Übungsblatt 2: Scheduling-Theorie und kooperativer Scheduler

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Zielsetzung

Sie prüfen ein Task-Set mit der Liu-und-Layland-Schranke und implementieren auf dem **NUCLEO-H755ZI-Q** (MCU STM32H755ZIT6, Cortex-M4) einen kooperativen Round-Robin-**Dispatcher**. Tasks sind endliche Schritte und kehren zurück. Es gibt keinen Assembler-Kontextwechsel und keine rekursiven Task-Aufrufe. Eine Messung am Oszilloskop ist optional und wird erst mit Protokoll als Messwert gewertet.

Aufgabe 1: Scheduling-Theorie (RMS-Schranke)

Ein Steuergerät soll drei periodische Tasks auf **einer** CPU verarbeiten (implicit deadlines $D_i = T_i$, Annahmen wie in der Vorlesung: unabhängig, voll präemptierbar, kein Overhead):

- τ_1: $C_1 = 1$ ms, $T_1 = 4$ ms
- τ_2: $C_2 = 2$ ms, $T_2 = 6$ ms
- τ_3: $C_3 = 1$ ms, $T_3 = 12$ ms

1. Berechnen Sie U.
2. Vergeben Sie RMS-Prioritäten.
3. Prüfen Sie $U \leq n(2^{1/n} - 1)$. Was folgt, wenn die Schranke **nicht** erfüllt wäre?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2: TCB für endliche Schritte

Definieren Sie im CM4-Projekt (`USER CODE`) einen TCB **ohne** Stackpointer: Funktionszeiger auf einen Step, Zustand, `wake_at`.

1. `TaskStep_t` als Zeiger auf `void (*)(void)`.
2. `struct MyTCB_t` mit `step`, `state`, `wake_at`.
3. `task_list[3]`, `rr_cursor`, `running_idx`.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 3: Dispatcher statt Rekursion

Implementieren Sie `dispatcher()` und `os_delay_ms()`. Der Dispatcher aktualisiert Timeouts, prüft höchstens N Einträge, ruft **einen** bereiten Step auf und bekommt die Kontrolle zurück. Danach wandert der Ringcursor. Ist niemand Ready: Idle (z. B. `__WFI()`), aber nur wenn eine Wake-Quelle läuft (HAL-Tick). Keine `do/while`-Suche ohne Abbruch.

`os_delay_ms` setzt `wake_at` und `ST_BLOCKED` und verlangt, dass der Step **sofort zurückkehrt**. Vergleich mit Wrap: `(int32_t)(now - wake_at) >= 0` für Abstände unter 2^{31} Ticks.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 4: Steps und zwei Messgrößen

Schreiben Sie `Task1_step`, `Task2_step`, `Task3_step`. Jeder Step arbeitet endlich und kehrt zurück (am Ende `os_delay_ms` nur mit sofortigem `return`).

1. `Task1_step`: Pin High, kurze Arbeit, Pin Low. Pin erst nach Belegung wählen (z. B. PA0).
2. In `main` die drei TCBs auf `ST_READY` setzen und `dispatcher()` aufrufen — nicht den Step aus einem anderen Step.
3. Am Oszilloskop **Pulsbreite** w und **Startabstand** P getrennt notieren. Eine konstante Verlängerung von Task 2 ändert P konstant und ist noch kein Jitter. Jitter entsteht bei **wechselnder** Last. Lehrwerte aus der Vorlesung nicht als Ihre Messung eintragen.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 5: Zwei verschiedene Fehler

1. **Kooperativ:** In `Task2_step` eine Endlosschleife ohne `return`. Beobachten Sie Task 1. Aktive Interrupts können weiterlaufen.
2. **Gedankenmodell präemptiv:** High bleibt dauerhaft Ready. Erklären Sie, warum ein `yield` von High, das High sofort wieder Ready lässt, Low nicht aus der Starvation holt.
3. Warum behebt „einfach FreeRTOS“ das nicht automatisch?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)